

Lab Code Errata

Modern Structured Outputs replacements for Companion v2 · Week 2 labs

WHAT THIS DOCUMENT FIXES

Companion v2 used the `prefill` pattern (assistant message ending with `{` to force JSON). That pattern was deprecated on Sonnet 4.6 and all newer models in late 2025. Six Week 2 labs return 400 errors as written. This errata supplies the corrected versions using **Structured Outputs** — the canonical modern pattern that Anthropic recommends and the CCA-F exam tests. **Use these versions instead of the labs in v2.**

SCOPE	Labs for Days 8–13 of v2 (W2 Mon–Sat)
API	<code>client.messages.parse(output_format=...)</code> · verified against current docs
MODEL	<code>claude-sonnet-4-6</code> (continues from v2)
SOURCE	platform.claude.com Structured Outputs docs
PREPARED	June 2026

01

What Changed & Why

In late 2025, Anthropic released **Structured Outputs** — a feature that constrains the model's decoder at the token level to follow a JSON Schema you supply. The model literally cannot emit invalid JSON. No parse errors, no schema drift, no retry needed. It's strictly better than the prefill technique for forcing structure.

With that release, Anthropic also **removed prefill support** on Sonnet 4.6 and all newer models (Opus 4.6, 4.7, 4.8, etc.). Sending a prefilled assistant message to any of these returns:

```
400 invalid_request_error:
  "Prefilling assistant messages is not supported for this model."
```

Companion v2's Week 2 labs were written using prefill — a pattern that works on Sonnet 4.5 and earlier but breaks on the model the labs tell you to use. This errata supplies the corrected versions using Structured Outputs, the canonical modern pattern.

EXHIBIT 1 • DECISION TABLE — WHICH PATTERN TO USE

YOUR GOAL	RIGHT PATTERN	API METHOD
Force JSON output matching a schema	Structured Outputs with Pydantic	<code>client.messages.parse(output_format=Model)</code>
Force a single label from a fixed set	Structured Outputs with <code>Literal[...]</code> field	<code>client.messages.parse(output_format=Model)</code>
Tool/function calls with typed args	Strict tool use	<code>tools=[...]</code> with <code>strict=True</code>
Free-form text with style constraints	System prompt instructions	<code>client.messages.create(system=...)</code>
Multi-turn chat (no structure needed)	Standard messages	<code>client.messages.create()</code>

EXAM IMPLICATION

The CCA-F exam launched in March 2026, after the structured-outputs release. The exam almost certainly tests Structured Outputs as the canonical pattern for forcing JSON, not prefill. **Memorize `messages.parse()` and `output_format` as the answer to JSON-extraction scenario questions.** Prefill is now legacy — useful background knowledge, but not what the exam will reward.

02

The Canonical Modern Pattern

Internalize this single example. Every Week 2 lab below is a variation of it.

```
from pydantic import BaseModel
from typing import Literal
from anthropic import Anthropic

load_dotenv()
client = Anthropic()

class Review(BaseModel):
    product_category: str
    sentiment: Literal["positive", "negative", "mixed"]
    positives: list[str]
    negatives: list[str]
    price_mentioned: float | None = None
    would_recommend: bool

response = client.messages.parse(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": f"Extract from this review: {review_text}" }],
    output_format=Review,
)

review = response.parsed_output
print(review.sentiment)          # already a Literal, type-checked
print(type(review).__name__)     # "Review"
```

FIVE THINGS THAT DIFFER FROM PREFILL

OLD (prefill)	NEW (structured outputs)
<code>client.messages.create(...)</code>	<code>client.messages.parse(...)</code>
Schema written into system prompt	Schema derived automatically from Pydantic model passed as <code>output_format</code>
Assistant prefill <code>{"role": "assistant", "content": "{ }"}</code>	No prefill. The decoder is constrained, not the prompt
Manual <code>json.loads(raw)</code> then <code>Model.model_validate(data)</code>	<code>response.parsed_output</code> returns a validated instance directly
Validation can fail → retry loop required	Validation cannot fail at JSON-shape level — retry loop largely obsolete

ONE GOTCHA

If you see a `TypeError` about Pydantic `BaseModel` subclassing when using `messages.parse()`, change your import from `pydantic import BaseModel` to `anthropic import BaseModel`. The SDK enforces its own `BaseModel` in some versions for type compatibility. Plain Pydantic works on most recent SDK versions; switch if needed.

L1

Classifier (Prompt Engineering Basics)

DAY

Day 8 • W2
Mon

FILE

labs\week-02\classifier_v0.py

CHANGE

Drop the three-version A/B (v0/v1/v2 with XML prefill).
Replace with one structured-outputs version.

```

from pydantic import BaseModel
from typing import Literal
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

EMAIL = """Hi,
Our entire payment system has been down for the last 3 hours. Customers are calling.
We are bleeding revenue. Please help ASAP. We pay $5000/month for this service.
– Jamie, COO, Acme Corp"""

class UrgencyClassification(BaseModel):
    urgency: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]

def classify(email: str) -> str:
    response = client.messages.parse(
        model="claude-sonnet-4-6",
        max_tokens=64,
        system="You classify support emails by urgency.",
        messages=[{"role": "user", "content": email}],
        output_format=UrgencyClassification,
    )
    return response.parsed_output.urgency

print("urgency:", classify(EMAIL))

```

WHAT THIS CHANGES

The v2 you couldn't run (XML prefill with `<urgency>`) becomes obsolete. Structured outputs with a `Literal` field is now the canonical pattern for single-label classification — stricter than prefill (the decoder physically cannot emit anything outside the Literal set) and shorter to write. **Exam-relevant takeaway:** when a scenario asks 'how do I guarantee Claude returns one of N labels?', the answer is now Structured Outputs with `Literal`, not `stop_sequences` + prefill.

L2

Few-Shot Classifier + Eval

DAY

Day 9 •
W2 Tue

FILE

labs\week-02\classify.py

CHANGE

Replace both `zero_shot` and `few_shot` functions. Examples now live in the system prompt as prose, not as XML-wrapped examples followed by prefill.

```
# labs\week-02\classify.py
from pydantic import BaseModel
from typing import Literal
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

class UrgencyClassification(BaseModel):
    urgency: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]

def zero_shot(email: str) -> str:
    response = client.messages.parse(
        model="claude-sonnet-4-6",
        max_tokens=64,
        system="Classify the urgency of the support email.",
        messages=[{"role": "user", "content": email}],
        output_format=UrgencyClassification,
    )
    return response.parsed_output.urgency

FEW_SHOT_SYSTEM = """Classify the urgency of the support email.
```

Examples:

- "URGENT: production payments completely down, losing revenue" → CRITICAL
- "Could you update my billing address when you have a moment?" → LOW
- "One of our team can't log in. Others fine. Look when you can." → MEDIUM
- "Admin dashboard unreachable for 30 min. Three staff blocked." → HIGH"""

```
def few_shot(email: str) -> str:
    response = client.messages.parse(
        model="claude-sonnet-4-6",
        max_tokens=64,
        system=FEW_SHOT_SYSTEM,
        messages=[{"role": "user", "content": email}],
        output_format=UrgencyClassification,
    )
    return response.parsed_output.urgency
```

WHAT THIS CHANGES

Your `eval.py` from this day is **unchanged** — it calls the two functions by name and compares strings. Re-run it after this update; few-shot should still beat zero-shot. If you've already labeled your gold set, that's also unchanged. **Note on the pedagogy:** few-shot examples in the system prompt are arguably less effective with structured outputs than they were with prefill (because the model is already constrained at the token level), but they still help on edge cases. Measure the delta with your eval to see if it matters for your data.

L3

JSON the Right Way

DAY

Day 10 •
W2 Wed

FILE

labs\week-02\extract.py

CHANGE

Complete rewrite. The schema lives in the Pydantic class only — no JSON Schema inlined into the system prompt, no prefill, no `json.loads`, no manual `model_validate`.

```
from pydantic import BaseModel
from typing import Literal
from anthropic import Anthropic
from dotenv import load_dotenv
```

```
load_dotenv()
client = Anthropic()
```

```
REVIEW = """Bought these for my morning runs. The cushioning is excellent and they're very
lightweight. However the laces came untied constantly and the toe box runs narrow — had to
size up. Overall happy, would recommend to other neutral-foot runners. Paid $130 on sale."""
```

```
class Review(BaseModel):
    product_category: str
    sentiment: Literal["positive", "negative", "mixed"]
    positives: list[str]
    negatives: list[str]
    price_mentioned: float | None = None
    would_recommend: bool
```

```
def extract(review_text: str) -> Review:
    response = client.messages.parse(
        model="claude-sonnet-4-6",
        max_tokens=512,
        system="Extract structured information from the product review.",
        messages=[{"role": "user", "content": review_text}],
        output_format=Review,
    )
    return response.parsed_output
```

```
result = extract(REVIEW)
print(result.model_dump_json(indent=2))
print(f"\nType is: {type(result).__name__}")
print(f"Would recommend: {result.would_recommend} (bool, not string)")
```

WHAT THIS CHANGES

Same conceptual goal as v2 (extract typed data from messy text), but the schema is the single source of truth — your Pydantic class is what generates the JSON Schema the decoder is constrained against, AND what your downstream code

consumes. No drift possible. The v2 lab told you to 'try and break it' by asking for `sentiment` as a list — that experiment doesn't work anymore because Claude physically cannot emit a list for a Literal field. Different kind of guarantee.

L4

Pydantic & Typed Extraction

DAY

Day 11 •
W2 Thu

FILE

labs\week-02\extract_typed.py

CHANGE

Add `Field(description=...)` via `Annotated` so the model sees per-field guidance through the schema.

```
# labs\week-02\extract_typed.py
# Note: this file is now essentially identical to extract.py from yesterday.
# The Pydantic class WAS the schema. With structured outputs, that's now also the
# schema the API uses. The "type your output" and "validate your output" steps that
# were separate yesterday are now the same step.

from pydantic import BaseModel, Field
from typing import Literal, Annotated
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

class Review(BaseModel):
    # Field descriptions are now embedded in the schema and used by the model.
    product_category: Annotated[str, Field(description="Generic category, e.g., 'shoes',
'headphones'")]
    sentiment: Annotated[Literal["positive", "negative", "mixed"], Field(description="Overall
sentiment")]
    positives: Annotated[list[str], Field(description="Specific positive phrases from the
review")]
    negatives: Annotated[list[str], Field(description="Specific negative phrases from the
review")]
    price_mentioned: Annotated[float | None, Field(default=None, description="Price in
dollars if mentioned")]
    would_recommend: Annotated[bool, Field(description="Whether the reviewer would
recommend")]

REVIEW = """Bought these for my morning runs. Cushioning excellent. Laces came untied
constantly. Sized up because toe box runs narrow. Would recommend. Paid $130."""

def extract(review_text: str) -> Review:
    response = client.messages.parse(
        model="claude-sonnet-4-6",
        max_tokens=512,
        system="Extract structured information from the product review.",
        messages=[{"role": "user", "content": review_text}],
        output_format=Review,
    )
    return response.parsed_output
```

```
result = extract(REVIEW)
print(result.model_dump_json(indent=2))
```

WHAT THIS CHANGES

The lesson of the day is now slightly different. In v2, the point was 'use Pydantic to validate the string-y output from Claude.' Now the lesson is 'use Pydantic to specify what you want — and field descriptions are how you communicate intent through the schema, not through prose in the system prompt.' Both `Annotated[...]` and the simpler `field: type = Field(...)` syntax work. **Test the validation:** ask for a confidence field as a `Literal["high", "medium", "low"]` and watch the model never deviate.

L5

The Retry Loop, Reconsidered

DAY

Day
12 •
W2 Fri

FILE

labs\week-02\extract_with_retry.py

CHANGE

Shrink the loop's scope. JSON-parse failures can't happen on Sonnet 4.6+; the loop now only catches *semantic* validation failures from custom `model_validator` rules.

```
# labs\week-02\extract_with_retry.py
#
# With Structured Outputs, the JSON-parse-failure case the retry loop was designed
# for can no longer happen on Sonnet 4.6+. The loop's surface area shrinks to:
#   (a) custom Pydantic validators that fail AFTER schema validation passes
#   (b) semantic checks you do yourself after parsing
#   (c) transient API errors (5xx, 429) – handled separately with backoff
#
# This is the modern equivalent: parse + post-validate + bounded retry on semantic
# failure only. Keep this pattern in your head for the exam – it's still tested.

from pydantic import BaseModel, model_validator
from typing import Literal
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

MAX_RETRIES = 3

class Review(BaseModel):
    product_category: str
    sentiment: Literal["positive", "negative", "mixed"]
    positives: list[str]
    negatives: list[str]
    price_mentioned: float | None = None
    would_recommend: bool

    @model_validator(mode="after")
    def semantic_check(self):
        # Example semantic rule that schema alone can't enforce:
        # if sentiment is positive, there must be at least one positive item.
        if self.sentiment == "positive" and not self.positives:
            raise ValueError("sentiment=positive but no positive phrases extracted")
        if self.sentiment == "negative" and not self.negatives:
            raise ValueError("sentiment=negative but no negative phrases extracted")
        return self

class ExtractionError(Exception):
    """Raised when extraction fails semantic validation after all retries."""
```

```

def extract(review_text: str) -> Review:
    messages = [{"role": "user", "content": review_text}]
    last_error = None

    for attempt in range(1, MAX_RETRIES + 1):
        try:
            response = client.messages.parse(
                model="claude-sonnet-4-6",
                max_tokens=512,
                system="Extract structured information from the product review.",
                messages=messages,
                output_format=Review,
            )
            return response.parsed_output # passes semantic check
        except ValueError as e: # raised by model_validator
            last_error = e
            print(f" [attempt {attempt} failed: {e}]")

            if attempt == MAX_RETRIES:
                break

            # Re-prompt with the specific semantic failure
            messages = [
                {"role": "user", "content": review_text},
                {"role": "user", "content": (
                    f"The previous extraction failed validation: {e}. "
                    f"Re-extract, paying attention to that constraint."
                )},
            ]

    raise ExtractionError(f"Failed after {MAX_RETRIES} attempts. Last: {last_error}")

```

WHAT THIS CHANGES

The whiteboard pattern you wrote up yesterday is still correct in structure (loop, max-tries, catch, re-prompt with error, raise on terminal failure) — the difference is what the loop is catching. Schema violations are now impossible at the API level. Semantic violations (a positive review with no listed positives, a refund > \$500 that wasn't escalated, etc.) are still possible and still benefit from the retry-with-correction pattern. **This is exactly how the exam will probe the pattern** — not 'fix invalid JSON' (no longer a thing) but 'fix output that's structurally valid but semantically wrong.'

L6

Prompt Caching

DAY

Day 13
• W2
Sat

FILE

labs\week-02\caching.py

CHANGE

Replace the `client.messages.create` + prefill call with `client.messages.parse` + `output_format`. Caching mechanics are unchanged — just drop the assistant prefill turn.

```
# labs\week-02\caching.py
import time
from pydantic import BaseModel
from typing import Literal
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

# A big static system prompt – caching only kicks in above 1024 tokens.
LONG_SYSTEM_BASE = """You extract structured information from product reviews.
Be thorough and capture specific phrases for positives and negatives."""

LONG_SYSTEM = LONG_SYSTEM_BASE + ("\nMore guidance: be precise. Quote specific phrases. " *
50)

class Review(BaseModel):
    product_category: str
    sentiment: Literal["positive", "negative", "mixed"]
    positives: list[str]
    negatives: list[str]
    price_mentioned: float | None = None
    would_recommend: bool

REVIEWS = [
    "Loved the running shoes. Light, cushioned. Paid $130.",
    "Coffee grinder works but is way too loud. $80.",
    "Watch band broke after a month. Returning. Was $50.",
    "Tent kept me dry through a storm – fantastic.",
    "Backpack zipper failed on day 2. $120 down the drain.",
]

def extract(review_text: str, use_cache: bool):
    system_blocks = [
        {
            "type": "text",
            "text": LONG_SYSTEM,
            **({"cache_control": {"type": "ephemeral"}} if use_cache else {}),
        }
    ]
```

```

response = client.messages.parse(
    model="claude-sonnet-4-6",
    max_tokens=400,
    system=system_blocks,
    messages=[{"role": "user", "content": review_text}],
    output_format=Review,
)
return response.usage

def run_batch(use_cache: bool):
    label = "WITH CACHE" if use_cache else "WITHOUT CACHE"
    print(f"\n=== {label} ===")
    total_input = total_cache_read = total_cache_create = 0
    t0 = time.time()
    for r in REVIEWS:
        u = extract(r, use_cache)
        total_input += u.input_tokens
        total_cache_read += getattr(u, "cache_read_input_tokens", 0) or 0
        total_cache_create += getattr(u, "cache_creation_input_tokens", 0) or 0
    print(f"  Uncached input: {total_input:,}")
    print(f"  Cache reads:      {total_cache_read:,}")
    print(f"  Cache creates:    {total_cache_create:,}")
    print(f"  Elapsed: {time.time() - t0:.2f}s")

run_batch(use_cache=False)
run_batch(use_cache=True)

```

WHAT THIS CHANGES

The prompt-caching mechanism (`cache_control: {"type": "ephemeral"}`) is model-version-agnostic and works the same way with structured outputs. The lab's lesson — measure cost reduction when re-sending a long static prefix — is unchanged. The 1024-token minimum is still the rule. **Exam-relevant pairing:** Anthropic's recommended pattern for production extraction pipelines is now Structured Outputs *plus* Prompt Caching — schema-guaranteed JSON with cached system prompt is the canonical 'cheap and reliable batch extraction' setup.

END**Notes & Forward****WHAT TO DO NOW**

- 01 Re-run each affected lab using the code in this errata. Confirm none return 400 errors.
- 02 Re-take the *Quiz on prompt engineering techniques* and *Quiz on prompt evaluation* on Skilljar with structured outputs as your mental model — it will resolve any "but how do I force JSON?" confusion.
- 03 Update your `notes\02-prompts.md` : replace any prefill-based notes with the Structured Outputs pattern. The retry-loop pseudocode you wrote up is still correct; just note that on modern models the loop catches *semantic* failures, not *parse* failures.
- 04 When I generate Weeks 3–4, the tool-use labs will use Structured Outputs natively. No more retroactive fixes.

THE PATTERN, ON ONE LINE

```
client.messages.parse(model=..., messages=..., output_format=PydanticModel) then read  
response.parsed_output . Memorize this; it answers most JSON-extraction exam questions.
```

WHY THE PREFILL TECHNIQUE STILL MATTERS (BRIEFLY)

Knowing prefill is useful background. It still works on Sonnet 4.5 and earlier (Claude 3.x too), so legacy codebases you might join will use it. Exam scenario questions may also reference it as a distractor — if a question describes a Sonnet 4.6 system and one option is "use prefill to force JSON," that's the wrong answer (400 error) and recognizing why is worth 1–2 points. The right answer is Structured Outputs.

See also: [Migration guide — Sonnet 4.6 breaking changes](#) and [Working with messages](#) in the docs.